In [ ]:
```
!pip install gensim
```

```
Collecting gensim
  Downloading gensim-4.4.0-cp312-cp312-manylinux_2_24_x86_64.manylinux_2_28_x8
6_64.whl.metadata (8.4 kB)
Requirement already satisfied: numpy>=1.18.5 in /usr/local/lib/python3.12/dist-
packages (from gensim) (2.0.2)
Requirement already satisfied: scipy>=1.7.0 in /usr/local/lib/python3.12/dist-p
ackages (from gensim) (1.16.3)
Requirement already satisfied: smart_open>=1.8.1 in /usr/local/lib/python3.12/d
ist-packages (from gensim) (7.5.0)
Requirement already satisfied: wrapt in /usr/local/lib/python3.12/dist-packages
(from smart_open>=1.8.1->gensim) (2.1.1)
Downloading gensim-4.4.0-cp312-cp312-manylinux_2_24_x86_64.manylinux_2_28_x86_6
4.whl (27.9 MB)
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 27.9/27.9 MB 20.0 MB/s eta 0:00:00
Installing collected packages: gensim
Successfully installed gensim-4.4.0
```

In [ ]:
```
import gensim
import numpy as np
import pandas as pd
from gensim.models import KeyedVectors
from sklearn.decomposition import PCA
import matplotlib.pyplot as plt
```

In [ ]:
```
import gensim.downloader as api

model = api.load("glove-wiki-gigaword-100")

print(len(model.key_to_index))
print(model['king'])
```

```
[==================================================] 100.0% 128.1/128.1MB downl
oaded
400000
[-0.32307  -0.87616   0.21977   0.25268   0.22976   0.7388   -0.37954
 -0.35307  -0.84369  -1.1113   -0.30266   0.33178  -0.25113   0.30448
 -0.077491 -0.89815   0.092496 -1.1407   -0.58324   0.66869  -0.23122
 -0.95855   0.28262  -0.078848  0.75315   0.26584   0.3422   -0.33949
  0.95608   0.065641  0.45747   0.39835   0.57965   0.39267  -0.21851
  0.58795  -0.55999   0.63368  -0.043983 -0.68731  -0.37841   0.38026
  0.61641  -0.88269  -0.12346  -0.37928  -0.38318   0.23868   0.6685
 -0.43321  -0.11065   0.081723  1.1569    0.78958  -0.21223  -2.3211
 -0.67806   0.44561   0.65707   0.1045    0.46217   0.19912   0.25802
  0.057194  0.53443  -0.43133  -0.34311   0.59789  -0.58417   0.068995
  0.23944  -0.85181   0.30379  -0.34177  -0.25746  -0.031101 -0.16285
  0.45169  -0.91627   0.64521   0.73281  -0.22752   0.30226   0.044801
 -0.83741   0.55006  -0.52506  -1.7357    0.4751   -0.70487   0.056939
 -0.7132    0.089623  0.41394  -1.3363   -0.61915  -0.33089  -0.52881
  0.16483  -0.98878 ]
```

In [ ]:
```
pairs = [
    ('doctor','nurse'),
```

```
    ('cat','dog'),
    ('car','bus'),
    ('king','queen'),
    ('boy','girl'),
    ('paris','france'),
    ('apple','orange'),
    ('teacher','student'),
    ('man','woman'),
    ('sun','moon')
]

for w1, w2 in pairs:
    print(w1, "-", w2, ":", model.similarity(w1, w2))
```

```
doctor - nurse : 0.7521509
cat - dog : 0.8798075
car - bus : 0.7372708
king - queen : 0.7507691
boy - girl : 0.91757303
paris - france : 0.74815863
apple - orange : 0.500704
teacher - student : 0.80833995
man - woman : 0.8323495
sun - moon : 0.6138352
```

In [ ]:
```
words = ['king','university','computer','india','music']

for word in words:
    print("\nTop words similar to", word)
    print(model.most_similar(word, topn=5))
```

```
Top words similar to king
[('prince', 0.7682328820228577), ('queen', 0.7507690787315369), ('son', 0.7020888328552246), ('brother', 0.6985775232315063), ('monarch', 0.6977890729904175)]

Top words similar to university
[('college', 0.8294212818145752), ('harvard', 0.8156033754348755), ('yale', 0.8113803267478943), ('professor', 0.8103784918785095), ('graduate', 0.7993000745773315)]

Top words similar to computer
[('computers', 0.8751984238624573), ('software', 0.8373122215270996), ('technology', 0.7642159461975098), ('pc', 0.7366448640823364), ('hardware', 0.7290390729904175)]

Top words similar to india
[('pakistan', 0.8370323777198792), ('indian', 0.780203104019165), ('delhi', 0.7712194323539734), ('bangladesh', 0.7661641240119934), ('lanka', 0.7639288306236267)]

Top words similar to music
[('musical', 0.8128045797348022), ('songs', 0.7978180646896362), ('dance', 0.7896507382392883), ('pop', 0.7862942218780518), ('recording', 0.7650765776634216)]
```

In [ ]:
```
print(model.most_similar(positive=['king','woman'], negative=['man'], topn=1))

print(model.most_similar(positive=['paris','india'], negative=['france'], topn

print(model.most_similar(positive=['teacher','hospital'], negative=['school'],
```

```
[('queen', 0.7698540687561035)]
[('delhi', 0.8654932975769043)]
[('nurse', 0.7798740267753601)]
```

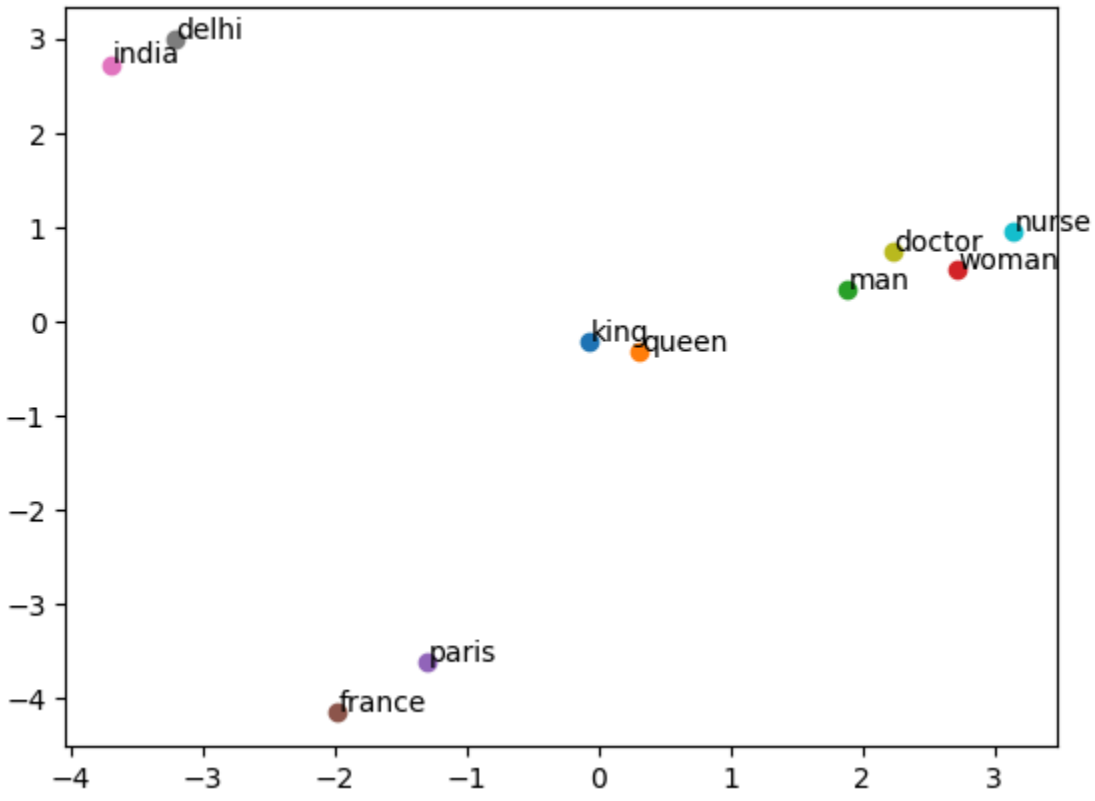
In [ ]:
```
words = ['king','queen','man','woman','paris','france','india','delhi','doctor

vectors = [model[w] for w in words]

pca = PCA(n_components=2)
result = pca.fit_transform(vectors)

plt.figure()
for i, word in enumerate(words):
    plt.scatter(result[i,0], result[i,1])
    plt.text(result[i,0]+0.01, result[i,1]+0.01, word)

plt.show()
```



In [ ]: